CIRCUSCIRCUSLIMES

Project Study Guide

A plain-language, printable guide to the DishCourse Flask forum: startup, routing, models, templates, MySQL persistence, direct messages, profile updates, and the request-response mental model.

CURRENT CODE VS. TARGET DESIGN This guide treats the files in the repository as the source of truth for what works now. The supplied target UML is used to explain the intended direction. Any mismatch is labeled explicitly.

Prepared from the project snapshot on August 21, 2026

Best use: read one section, cover the answer, and explain it aloud in your own words.

How to use this guide

Do not try to memorize every line. Learn the path a request takes, the job of each module, and the database relationships. Once that mental map is stable, individual functions become much easier to understand.

The two-layer reading method

Layer	Question to ask	CircusCircusLimes answer
1. Purpose	What does this thing do?	It is a Flask discussion forum branded as DishCourse. People can create accounts, browse subforums, publish posts, comment, react, send direct messages, and manage profile details.
2. Parts	What are the main pieces and how do they connect?	Flask receives requests; blueprints route them to controller functions; SQLAlchemy reads/writes MySQL; Jinja templates and CSS build the response; Flask-Login tracks the current user.

Fast mental model

```
Browser → Flask app → matching route/controller → validation and business logic →
SQLAlchemy/MySQL → Jinja template or redirect → browser
```

ONE-SENTENCE EXPLANATION CircusCircusLimes is a server-rendered Flask forum organized into feature blueprints, backed by MySQL through SQLAlchemy, with HTML generated from Jinja templates.

Reading map

If you want to understand...	Start here
How the app boots	`forum/app.py`, then `forum/__init__.py`, then `config.py`
URLs and actions	The seven blueprint files plus the `/` route in `forum/app.py`
Stored information	`forum/models.py`
Validation	`forum/user.py` and validators at the bottom of `forum/models.py`
What users see	`forum/templates/` and `forum/static/`
Containers and MySQL	`compose.yaml`, `Dockerfile`, `config.py`

1. Startup: how the application comes alive

Startup components

- **Dockerfile** builds a Python 3.11 image, installs requirements, copies the project, exposes port 5000, and starts Gunicorn.
- **compose.yaml** starts two services: the web application and MySQL 8.4. It passes database environment variables to the web container.
- **forum/app.py** imports and calls `create_app()`, creates the Flask-Login manager, defines the home route, and seeds starter subforums if the database is empty.
- **forum/__init__.py** contains the app factory, applies configuration, registers seven blueprints, connects SQLAlchemy, and creates tables.
- **config.py** builds the MySQL connection URL from `DB_USER`, `DB_PASSWORD`, `DB_HOST`, and `DB_NAME`.

Exact startup sequence

1. Gunicorn imports the object named `app` from `forum.app`.

2. Importing `forum/app.py` runs `app = create_app()`.

3. `create_app()` constructs Flask, loads `config.Config`, and registers the posts, comments, reactions, messages, settings, profile, and routes blueprints.

4. `db.init_app(app)` connects the SQLAlchemy extension to this Flask app.

5. Inside an application context, `db.create_all()` creates any missing tables, then the app object is returned.

6. Back in `forum/app.py`, Flask-Login is attached. `load_user()` tells Flask-Login how to reload a user by ID.

7. A second app context runs table creation again and seeds five initial subforums if none exist.

8. Gunicorn listens on `0.0.0.0:5000` inside the web container; Docker maps the host's `APP_PORT` to that port.

> **WATCH OUT** `db.create_all()` appears twice. It is normally harmless because it creates missing tables rather than recreating existing ones, but it is redundant. It also is not a migration system: schema changes should eventually use a migration tool.

How does Flask start?

In production-style container startup, Gunicorn imports `forum.app:app`. During that import, Python executes `app = create_app()`. For local development, the Flask command uses `FLASK_APP = 'forum.app'` and finds the same app object.

How does create_app() execute?

It executes as a normal Python function call during module import. It builds and returns the configured Flask object.

How are blueprints registered?

Each feature file creates a `Blueprint` object. `create_app()` imports those objects and calls `app.register_blueprint(...)` for each one.

2. Routing: URL to controller function

A route connects an HTTP method and URL pattern to a Python function. Flask checks its URL map, selects the matching function, and calls it. In this app, route functions are the controllers: they interpret input, apply business rules, use models, and choose a response.

Complete live route inventory

Method / URL	Endpoint function	Module	Login?
GET /	index	app.py	No
POST /action_login	action_login	routes.py	No
GET /action_logout	action_logout	routes.py	Intended yes*
POST /action_createaccount	action_createaccount	routes.py	No
GET /subforum?sub=ID	subforum	routes.py	No
GET /loginform	loginform	routes.py	No
GET /about | /contact | /house-rules	about / contact / house_rules	routes.py	No
GET /addpost?sub=ID	addpost	posts.py	Yes
GET /viewpost?post=ID	viewpost	posts.py	No; private posts require login
POST /action_post?sub=ID	action_post	posts.py	Yes
POST /action_comment?post=ID	comment	comments.py	Yes
POST /action_reaction	react	reactions.py	Yes
GET /messages	inbox	messages.py	Yes
GET /messages/<username>	conversation	messages.py	Yes
POST /action_send_message	send_message	messages.py	Yes
GET /settings	settings_page	settings.py	Yes
POST /action_change_password	change_password	settings.py	Yes
GET /profile	profile_page	profile.py	Yes
POST /action_update_username	update_username	profile.py	Yes
POST /action_update_email	update_email	profile.py	Yes
POST /action_update_avatar	update_avatar	profile.py	Yes

* In `routes.py`, `@login_required` is placed above `@rt.route` for logout. Decorator order matters; the conventional and clearly protected form places `@rt.route` first and `@login_required` directly above the function.

How HTTP requests are routed

Example: a browser submits a heart reaction. The form sends `POST /action_reaction`. Flask's URL map points that request to `reactions.react`. The controller validates the reaction, queries the post and existing reaction, mutates the database, commits, then returns a redirect to the post page.

DIAGRAM CORRECTION The supplied target UML shows `/action_react?post=<id>`. The current code actually uses `POST /action_reaction` and receives `post_id` plus `reaction_type` from form data.

3. Data model: what is stored and how it connects

A SQLAlchemy model is a Python class mapped to a database table. A model object represents one row. Columns store values, foreign keys connect rows, and relationships let Python navigate those connections.

Model	Important fields	Relationships / behavior
User	id, username, password_hash, email, admin, avatar_filename	Has posts, comments, reactions, sent_messages, received_messages. Passwords are hashed; `check_password` verifies them.
Post	id, title, content, postdate, user_id, subforum_id, is_public	Belongs to one user and one subforum; has many comments and reactions. Computes a cached relative-time string.
Comment	id, content, postdate, user_id, post_id	Belongs to one user and one post. Also computes relative time.
Subforum	id, title, description, parent_id, hidden	Can have child subforums and many posts. `parent_id` makes the hierarchy self-referential.
Reaction	id, reaction_type, user_id, post_id	Belongs to one user and one post. A unique constraint permits only one reaction per user/post pair.
Message	id, content, sentdate, is_read, sender_id, recipient_id	Has two separate relationships to User: sender and recipient. Backrefs expose sent and received messages.

Relationship map

```
User 1 → many Posts User 1 → many Comments User 1 → many Reactions Subforum 1 → many Posts
Post 1 → many Comments Post 1 → many Reactions User 1 → many sent Messages; User 1 → many
received Messages
```

What SQLAlchemy does here

- `Model.query.filter(...)` builds and executes SELECT queries.
- `db.session.add(object)` stages a new row; appending through a relationship can also associate new objects.
- `db.session.delete(object)` stages deletion.
- `db.session.commit()` completes the transaction and makes changes persistent.
- A `ForeignKey` stores the linked row's ID. A `relationship`/`backref` gives convenient Python attribute access.

TARGET UML MISMATCHES There is no live `UserSettings` class. Theme and email-notification fields are planned, not implemented. Post `format` and `media_url` are also absent. The live Message field is named `content`, not `body`. The live Reaction field is `reaction_type`, not `type`.

What are the classes and where are they?

The six database model classes—User, Post, Subforum, Comment, Reaction, and Message—are all in `forum/models.py`. Flask itself is a class instantiated in `create_app()`; each Blueprint is also an object created in its feature module.

4. User utilities and validation

Validation answers: “Is this input shaped correctly?” Account checks answer: “Does this value already exist?” These are helper functions because several controllers can reuse them.

Helper	Rule / result	Used for
valid_username	4-40 characters; letters, numbers, ! @ # % &	Account creation and username update
valid_password	6-40 characters; same allowed character set	Password change (account creation currently does not enforce it)
valid_email	Simple pattern requiring text@text.domain	Profile email update
username_taken	Queries User and returns a row or None	Prevent duplicate usernames
email_taken	Queries User and returns a row or None	Prevent duplicate emails
valid_avatar_filename	Allows png, jpg, jpeg, gif, webp	Avatar upload
avatar_extension	Returns lower-case extension	Build stored avatar filename
valid_title / valid_content	Title >4 and <140; content >10 and <5000	New posts
valid_comment	Trimmed content 1-1000 characters	New comments
valid_message	Trimmed content 1-2000 characters	Direct messages

Password handling

The application never needs to store the original password. The User constructor calls Werkzeug's `generate_password_hash()`. Login and password change call `check_password_hash()`. A successful password change replaces the stored hash and commits.

> **IMPORTANT GAP** Account creation currently has the `valid_password(password)` check commented out. A password that would be rejected on the settings page may still be accepted during signup.

Avatar safety checks

The controller cleans the original name with `secure_filename`, allows only selected extensions, checks that the stream is at most 2 MB, creates a random server-side filename, saves into `forum/static/images/avatars/`, commits that filename to User, and removes the old avatar file.

The global `MAX_CONTENT_LENGTH` is about 2.5 MB, providing an earlier request-size backstop.

5. Presentation: templates, shared layout, and CSS

The app uses server-side rendering. A controller calls `render_template()`, Jinja fills placeholders using Python data, and Flask returns completed HTML to the browser.

How templates receive data

Controller call	Template receives
`render_template('subforums.html', subforums=subforums)`	A query result containing top-level subforums
`render_template('viewpost.html', post=..., comments=..., like_count=...)`	Post, breadcrumb path, comments, three counts, and the current user's reaction
`render_template('conversation.html', other=..., thread_messages=...)`	The conversation partner and ordered message history

Shared presentation structure

- Feature templates use or extend `layout.html`.
- The layout includes `header.html`, `sidebar.html`, and `right_sidebar.html` so common UI is not duplicated.
- It loads local `bootstrap.min.css`, Bootstrap Icons from a CDN, and custom `style.css`.
- It displays ordinary validation errors and flashed success/error messages.
- The messages blueprint adds `unread_message_count` to every template through an app context processor.

Template vocabulary

Jinja syntax	Meaning
`{{ value }}`	Print a value into HTML (normally escaped).
`{% if ... %}`	Conditionally include markup.
`{% for ... %}`	Repeat markup for each item.
`{% include 'header.html' %}`	Insert a shared partial.
`{% block body %}`	Define the region a child template fills.

PRESENTATION VS. BUSINESS LOGIC Templates should mainly display decisions already made. Rules such as “cannot message yourself,” “private posts require login,” and “one reaction per user/post” belong in controllers and models, not in CSS or HTML.

6. Database and persistence

What are the database components?

MySQL 8.4 is the database server; PyMySQL is the Python driver; SQLAlchemy maps Python models to SQL; Flask-SQLAlchemy connects that mapping to Flask; `config.py` builds the connection URL; and Docker Compose supplies credentials and persistent storage.

Where does persistence occur?

Application code commits through `db.session.commit()`. MySQL writes data files under `/var/lib/mysql` inside the database container, backed by the named Docker volume `dishcourse_mysql_data`. The volume lets data survive replacement of an individual container.

Database environment variables

Application variable	Compose source	Meaning
DB_USER	MYSQL_USER	MySQL account used by Flask
DB_PASSWORD	MYSQL_PASSWORD	That account's password
DB_HOST	db	Compose service hostname for MySQL
DB_NAME	MYSQL_DATABASE	Database/schema name
APP_PORT	Host .env / shell	Host port mapped to container port 5000
DB_PORT	Host .env / shell	Optional host-only mapping to MySQL port 3306

How the database is created

Compose starts MySQL with `MYSQL_DATABASE`. When the Flask app starts, `db.create_all()` sends CREATE TABLE statements for missing model tables. If the Subforum table is empty, `init_site()` inserts the default categories and commits them.

> **CURRENT, NOT OLD README** The README still contains historical SQLite notes, but the current `config.py` uses `mysql+pymysql` and `compose.yaml` defines MySQL. For the current code, MySQL is the correct answer.

Transaction pattern

```
validate input → create or change model objects → db.session.add() if needed →
db.session.commit() → redirect
```

7. Direct messages: complete feature walkthrough

Where is the messages blueprint?

It is created in `forum/messages.py` as `messages = Blueprint('messages', __name__)` and registered in `forum/__init__.py`.

What changed for direct messages?

- Registered the new messages blueprint in the app factory.
- Added the Message model, its sender/recipient foreign keys and relationships, read status, timestamp, and relative-time method.
- Added message-content validation.
- Added inbox, conversation, and send-message controller routes.
- Added `messages_inbox.html` and `conversation.html`.
- Added navigation and unread-count UI, plus a “message this author” entry point on post pages.
- Added message styling to `style.css`.

Routes

Route	Purpose
GET /messages	Query all messages involving the current user, collapse them into one summary per conversation partner, sort by newest, and render the inbox.
GET /messages/<username>	Load the two-way thread, reject nonexistent/self conversations, mark received unread messages as read, and render the conversation.
POST /action_send_message	Validate recipient and content, create Message, attach sender and recipient, commit, and redirect to the conversation.

Request-response flow for sending a DM

1. A logged-in user submits the conversation form with `recipient` and `content`.

2. Flask matches `POST /action_send_message` to `messages.send_message`. The login decorator blocks anonymous access.

3. The controller trims values and queries User by username.

4. It rejects a missing recipient, a self-message, or content outside 1-2000 characters.

5. It creates `Message(content, now)`, sets `sender` and `recipient`, adds it, and commits.

6. It redirects to `/messages/<username>`. The browser follows with a GET.

7. The conversation controller reloads the ordered thread and renders `conversation.html`.

WHY TWO USER FOREIGN KEYS? A message points to User twice for different roles. Explicit `foreign_keys` arguments remove ambiguity and let SQLAlchemy expose `message.sender` and `message.recipient`.

8. Profile and settings: complete feature walkthrough

What is the profile blueprint?

`forum/profile.py` owns the page and three independent update actions: username, email, and avatar. It is registered in `create_app()`; every profile route requires login.

What changed for profile?

- Added `avatar_filename` to User. The database stores only the generated filename; the image itself lives in the static avatar directory.
- Added the profile blueprint and its four routes.
- Added email and avatar validation helpers plus a 2 MB avatar limit.
- Added `profile.html`, adjusted the settings page so it focuses on password, and updated header/sidebar links.
- Added avatar rendering and profile styles; updated request-size configuration.

Profile routes

Route	Purpose
GET /profile	Render the current user's editable profile.
POST /action_update_username	Validate format and uniqueness, update User.username, commit, flash result.
POST /action_update_email	Validate format and uniqueness, update User.email, commit, flash result.
POST /action_update_avatar	Validate and save image, update User.avatar_filename, commit, then remove the previous file.

Request-response flow: changing profile data

The page deliberately uses separate forms. That means the server receives one focused change at a time.

```
Browser form → matching POST route → @login_required → read form/file input → validate → update current User (and possibly save file) → commit → flash message → redirect to GET /profile → render updated values
```

Profile vs. settings

Profile owns avatar, username, and email. Settings currently owns only password change. The target UML's separate UserSettings record is future architecture, not current behavior.

FILE/DB CONSISTENCY NOTE The avatar controller commits the new filename before deleting the old file. This avoids losing the old file before the database update succeeds, though a filesystem failure after commit could leave an unused old file.

9. Core concepts in plain English

Flask

A lightweight Python web framework. It receives HTTP requests, maps them to Python functions, and returns responses.

Server

A program that listens for requests and sends responses. Here, Gunicorn runs the Flask app; MySQL is a separate database server.

Tech stack

Python 3.11, Flask, Gunicorn, Flask-Login, Flask-SQLAlchemy/SQLAlchemy, PyMySQL, MySQL 8.4, Jinja2, HTML/CSS, Bootstrap, Docker and Docker Compose.

Blueprint

A package of related Flask routes and behavior. It keeps one large application organized by feature.

Controller

The route function that coordinates a request: read input, apply rules, call models, and choose a template or redirect.

Business logic

Rules that make the forum behave correctly—for example validation, privacy checks, reaction toggling, and preventing self-messages. It mostly lives in route functions and reusable validators; the uniqueness constraint is in the model.

Decorator

A wrapper written with `@` that changes function behavior. `@route` registers a URL; `@login_required` blocks anonymous users.

API

A defined way for software parts to communicate. Flask's functions are an API for building web apps; the browser also interacts with URL endpoints.

REST API

An HTTP API organized around resources and standard methods, often returning JSON. This project is mainly a server-rendered web app with action-style form routes, not a strict REST API.

rt in rt.route

`rt` is the variable holding the blueprint named `routes`. `rt.route(...)` registers a function on that blueprint.

Module

A Python file that can be imported. Examples: posts.py, messages.py, models.py, and user.py.

Template

An HTML file with Jinja placeholders and control statements. Flask combines it with data to produce final HTML.

Persistence

Data continues to exist after a request or process ends. MySQL plus its Docker volume provide persistence.

Virtual environment

An isolated set of Python packages for one project, preventing dependency conflicts with other projects.

Port

A numbered network doorway. Port 5000 is where the web process listens inside its container.

localhost

The current computer. `http://localhost:5000` asks your own machine for an HTTP response on port 5000.

Docker

A tool that packages applications and dependencies into containers. Compose describes how the web and database containers run together.

10. Feature flow charts and request-response sequences

Login

```
POST form → routes.action_login → query User by username → verify hash → login_user(user) → redirect / → render subforums
```

Create post

```
GET /addpost?sub=ID → login check → render form → POST /action_post?sub=ID → validate title/content → create Post → connect User + Subforum → commit → redirect /viewpost?post=ID
```

View a post

```
GET /viewpost?post=ID → find Post → enforce private-post rule → load breadcrumb, comments, counts, user's reaction → render viewpost.html
```

Comment

```
POST /action_comment?post=ID → login check → validate ID and content → create Comment → connect User + Post → commit → redirect to post
```

Reaction toggle

```
POST /action_reaction → login check → validate type → find Post → find user's existing Reaction → same type: delete | different type: update | none: create → commit → redirect
```

Password change

```
POST /action_change_password → login check → verify current password → validate and compare new password → hash new password → commit → flash success → redirect settings
```

General request-response sequence

Stage	What happens
1. Request	The browser sends method, URL, query string, form fields, cookies, and possibly a file.
2. Routing	Flask finds the route whose method and URL pattern match.
3. Authentication	If present, `login_required` checks session state before the controller proceeds.
4. Controller	The route function parses and validates input and applies business rules.
5. Model/database	SQLAlchemy queries or changes mapped objects; commit makes writes persistent.
6. Response	The controller renders HTML, redirects, or returns an error string.
7. Browser	The browser displays HTML or follows the redirect with another request.

11. Current architecture vs. target architecture

The target diagram is useful as a roadmap, but the current repository has moved partway toward it. Use this page to avoid blending present facts with planned work.

Area	Current repository	Target / planned
Blueprints	7 registered: posts, comments, reactions, messages, settings, profile, routes	Target labels the legacy routes blueprint as auth.py; the current file is still routes.py and also owns subforum/static-page routes.
Database	MySQL through PyMySQL and SQLAlchemy	Achieved.
Comments	Many comments per post	Achieved.
Reactions	Like/dislike/heart with one reaction per user/post	Achieved, but diagram route/name details differ.
Messages	Direct messages, inbox, threads, read status	Achieved; model uses content/sentdate naming.
Profile	Avatar, username, email updates	Achieved.
Settings	Password change only; no UserSettings table	Theme/email notifications via UserSettings are planned.
Visibility	Post.is_public; private view blocked for guests	Mostly achieved. Subforum listings should also be reviewed to ensure private titles/content are not exposed.
Rich content	Plain text content; media/markdown fields absent	Post format and media_url planned.
UI	Shared layout, logo, footer, Bootstrap/custom CSS	Largely achieved.

Important files and their jobs

File/folder	Job
forum/app.py	Runtime entry point, login manager, home page, seed data
forum/__init__.py	Application factory and blueprint/SQLAlchemy registration
config.py	MySQL URL and Flask configuration
forum/models.py	Database models and content validators/helpers
forum/user.py	Account and avatar validators/checks
forum/*.py feature modules	Controllers grouped by feature
forum/templates/	Jinja presentation files
forum/static/	CSS, Bootstrap, logo, uploaded avatars
compose.yaml / Dockerfile	Container startup and web/database wiring

12. Comprehension check

Try these without looking back. Then use the answer key on the next page.

1. What line causes the Flask application factory to execute?
2. Name all seven registered blueprints.
3. What is the difference between a foreign key and a relationship?
4. Which route creates a reaction, and what three reaction values are accepted?
5. Why does Message need two foreign keys to User?
6. What makes a database change persistent?
7. Where do avatar bytes live, and what does the User table store?
8. Which file supplies shared page layout and which file supplies custom styling?
9. Explain the complete request path for sending a direct message.
10. Which target-UML model does not exist in the current code?
11. Is this project primarily a strict REST API? Why or why not?
12. What does Docker's named MySQL volume protect against?
13. What does `@login_required` do?
14. Which current signup validation rule is commented out?
15. Explain `http://localhost:5000` word by word.

Notes:

Answer key

1. `app = create_app()` in `forum/app.py`.

2. posts, comments, reactions, messages, settings, profile, and routes.

3. A foreign key is the stored ID that enforces a database link. A relationship gives convenient Python navigation between linked objects.

4. `POST /action_reaction`; like, dislike, and heart.

5. Each message has two User roles: sender and recipient.

6. A successful `db.session.commit()` completes the transaction.

7. Bytes live in `forum/static/images/avatars/`; User stores only `avatar_filename`.

8. `forum/templates/layout.html` and `forum/static/style.css`.

9. Form POST → matching route/login check → recipient/content validation → create and link Message → add/commit → redirect → conversation GET/query/render.

10. UserSettings.

11. No. It is mainly a server-rendered HTML app with action-style form endpoints and redirects, not a resource-oriented JSON API.

12. It keeps MySQL data when the database container itself is replaced or recreated.

13. It requires an authenticated Flask-Login user or redirects to the configured login view.

14. The password-format check in account creation.

15. http = protocol; localhost = this computer; 5000 = the network port; / = the root path.

Teach-back prompts

- Draw the request-response sequence from memory and label where authentication, validation, database work, and rendering occur.
- Pick one blueprint and explain its routes, input, business rules, models, and output.
- Explain one current-vs-target mismatch without looking at the guide.
- Describe how one user can own posts, comments, reactions, sent messages, and received messages.

Source notes and accuracy boundaries

This guide was built from the live repository files, the supplied comprehension-questions PDF, the supplied updated target-architecture image, and recent project history for the direct-message and profile feature commits.

Primary project evidence

- Startup/configuration: `forum/app.py`, `forum/__init__.py`, `config.py`, `Dockerfile`, `compose.yaml`, `requirements.txt`.
- Routing/business logic: `forum/routes.py`, `posts.py`, `comments.py`, `reactions.py`, `messages.py`, `settings.py`, and `profile.py`.
- Models/helpers: `forum/models.py` and `forum/user.py`.
- Presentation: `forum/templates/` and `forum/static/`.
- Feature history: commits `750b831` (direct messages), `762aaf1` (settings), and `1db9ff1` (profile).

BOUNDARY “Current” means visible in this repository snapshot, not necessarily fully tested in a running environment. “Target” means shown or implied in the updated architecture diagram/README feature list. Where names or behavior disagree, the current code is described first.

Five facts worth memorizing

- The app factory is `create_app()` in `forum/__init__.py`.
- Routes are controllers grouped into seven blueprints, plus the root route in `app.py`.
- Six current SQLAlchemy models live in `models.py`; UserSettings is planned only.
- MySQL persistence happens through SQLAlchemy commits and the Docker named volume.
- A request usually ends by rendering a Jinja template or redirecting the browser to another GET.